

Project 3 Tom & Jerry AI Agent Implementation:

Evan Litzer

I. INTRODUCTION

This report entails the updated planner file I have created for project three. This file has significantly boosted Tom's planning capabilities to correctly handle probabilistic transitions and its uncertain movements. Expanding upon the second assignment, I created an alpha-beta pruning algorithm that factors in the direction manipulation using a random probability distribution over the three rotation outcomes. Most prominently though, I implemented an uncertainty system that adjusts confidence in actions based on Jerry's distance from Tom. I also made sure to emphasize obstacle avoidance through mobility and safety checks, along with tuning evaluation with dead-end possibility and threatening distance from Tom. With all of these changes, a more intelligent agent arises in my Tom file, which avoids Spike and catches Jerry at a higher rate.

II. OVERALL AGENT DESIGN & OBJECTIVE

A. Q-Learning for Chasing Jerry and Avoiding Spike

My Tom planner file uses Q-learning as a way to guide his future actions in the newly instantiated probabilistic-transition environment of the grid. At all times, the agent is multitasking between avoiding the Spike agent and capturing the Jerry agent. Q-learning allows Tom to learn the long term value of his different possible actions by interacting with each of the game grids. Each of the states of the game is configured with a set of position vectors that represent the starting position of the three agents. These are then pointed to a set of Q-values that represent the expected reward for each possible move. This process of learning teaches Tom to improve his decision-making and choice of action in scenarios that present difficulty, which this rotation of direction probability vector is presenting with movement distortion.

B. Learns and Stores Knowledge in Q-table

The agent trains by running countless simulations of grid-interactions with its game environment. Tom observes the positions of the agents and chooses an appropriate action using a greedy strategy, and then receives the reward based on the action-outcome. These simulated experiences then iterate and update the Q-values that are connected with each state-action pair through optimization of the Bellman equation. These values are then stored in the Q-table, which is a dictionary that connects state to action values. As the planner learns and iterates through the grid, it gains knowledge about the game information and what Tom should and should not do

based on the game-scoring system. This table and learning process allows the Tom agent to learn what behavior is good across every grid layout.

III. PROBABILITY TRANSITIONS

A. Modeling Probability Transitions

To model the probabilistic transitions that are assigned to each game-grid instance, I simulate Tom's favorable movement being altered if the movement distortion is not straight. The directions that the assignment provides is left, right, and straight—with left and right being the real alterations to the movement. To address these rotations, I create a distribution of the probability over the outcomes of each rotation for all of the move directions.

I obtain the probabilities through the `get_dynamic_probs()` method which are calculated based on Tom's distance from Spike. When Spike is closer to Tom, the model increases the weight on side rotations and less on the straight to account for the possible rotating distortions that could lead Tom into Spike.

The always changing transition probabilities are integrated in both the learning and moving parts of the agent's execution. In the train method, the agent uses the probabilities to simulate which direction Tom will move in, which then allows the Q-table to learn the best moves even under the stochastic rotation. By this modeling that considers rotation, the agent learns more a more effective strategy that is prepared for the probability transition.

B. Uncertainty Adaptation

The probability that Tom goes straight, left, or right is adjusted based on his distance to Spike. When Spike is further away from Tom, Tom will be more confident in his decision and put more weight on the straight probability rotation. When Spike is close to Tom, the uncertainty will increase through higher weights on the left and right rotation probabilities. This is a very conservative approach to this assignment, as Tom will be overly-cautious when in the presence of Spike. This uncertainty adjustment is present through all phases of training and movement.

IV. Q-LEARNING

A. States & Q-Table Interaction

Each state is constructed through the positions of the three agents: Tom, Jerry, and Spike. They are combined into a tuple using the `state_key()` helper method. This is then the key in the Q-table, which is implemented as a `defaultdict` that maps the states to the arrays of the Q values, as there is one for each

of the three distortion directions. The Q-values represent the expected reward of a specific action from the state. This then allows the agent to store and update its knowledge on what action Tom should and should not take when Spike has different levels of presence.

B. Q-Learning Training

The agent is trained through simulated game scenarios where Tom interacts with the grid environment in the `train()` function. The current state (agent positions) is the home base for the agent to use an epsilon-greedy strategy to explore from in the `choose_action()` method. The action that is chosen with the highest expected reward is then returned, but it is then rotated using the transition probabilities from `get_dynamic_probs()`.

Once the rotation is applied, the agent then transitions to a new state and receives a reward calculated by the `get_reward()` function. Then it updates the Q-value for the state-action pair using the Bellman equation applied in the `update()` function.

Over many simulated scenarios, the agent eventually learns which actions lead to the highest rewards, which enable it to construct an effective policy for catching Jerry and avoiding Spike. This learning is carried out through the Q-table.

V. REWARDING

The agent uses a layered reward system that guides learning and provides the agent with the best calculated move in any situation. I calculate this reward in the reward function `get_reward()`. Tom receives a high reward for catching Jerry and a large flag for being caught by Spike, which is done through comparing positions. After these two checks, there are many factors that play a role in deciding whether a move is beneficial for Tom.

An example of this is Tom being rewarded for reducing his distance to Jerry and being penalized for getting dangerously close to Spike with no immediate presence of Jerry. I also have bonuses for proximity to Jerry and penalties for mobility and how many obstacles are nearby. This reward calculation helps Tom pursue Jerry while being aware of Spike's pursuit of Tom.

The weights on each of the factors have been carefully tuned and tested for performance. My goal was to balance Tom's aggressiveness and caution to have him win games but also avoid Spike and obstacles. It is very similar to the reward function I implemented in assignment 2, as the probability transitions play no role here.

VI. CONCLUSION

The agent's performance is unfazed by the probability transition that occurs in the main file. It correctly prepares for all cases of distortion and ensures that the agent stays on its path of avoiding Spike and catching Tom. It simulates countless moves and updates the Q-values accordingly. The

rewarding strategy balances aggression, mobility, and defense to contribute to a balanced strategy emphasizing on keeping the agent alive. Depth is kept lower but is ramped up when the agent is near its mortal enemy.